

Software Engineering Intro

Condensed Study Notes

Generated: 2026-05-25 00:11 UTC

Course Contents

This course is going to explore different aspects of how we build software, which is the set of instructions that tells computers what to do. We'll be looking at how to plan and structure these instructions in a way that makes sense and is easy to manage.

We'll start by thinking about the big picture of how software is put together, like planning the layout of a whole city before building individual houses. This is called **Software Architecture** (the overall structure of a software system).

Then, we'll dive into **Design Patterns** (common, reusable solutions to everyday problems in software design). Think of these like a set of well-tested blueprints for common building elements, like a standard way to design a sturdy door or a comfortable staircase. These patterns help us solve recurring issues in software development efficiently.

We'll also learn about **OO Analysis & Design**. 'OO' stands for Object-Oriented, which is a way of thinking about software where we model things in the real world as 'objects' – like a 'car' object that has properties (color, speed) and actions (accelerate, brake). This helps organize complex systems.

Finally, under 'Software Design', we'll touch upon **Design for re-use**. This means building software components that can be used in many different projects, like using pre-fabricated bathroom pods in construction instead of building each one from scratch every time.

Next, we'll look at **Using APIs**. An **API** (Application Programming Interface) is like a menu at a restaurant. It tells you what dishes (functions or services) are available and how to order them, without you needing to know how the kitchen (the other software) actually makes the food. We'll explore **API programming** (writing code that uses these menus) and tools like **Class browsers** (which help you see and understand the available 'dishes' or functions in an API) and other related tools.

We'll also cover **Component-based computing**. This is like building with LEGOs. Instead of building everything from the ground up, we assemble pre-made 'components' (independent pieces of software) to create a larger application. This makes development faster and more flexible.

Lastly, we'll examine **Software tools and Environment**. This involves the various helper programs and systems that make building software easier. This includes **Requirements analysis and design modeling tools** (software that helps you plan what the software should do and how it should look, like a digital whiteboard and sketching tool for architects), **Testing tools** (programs that check if your software works correctly, like a quality inspector for a factory), and **Tool integration mechanisms** (ways to make all these different tools work together smoothly, like having a central hub that connects different specialized machines in a workshop).

The Term Software

Imagine you have a recipe book and a set of kitchen tools. The recipe book contains instructions on how to make a dish, and the tools help you chop, mix, and cook.

Software is very similar! It's a collection of instructions that tell a computer what to do. These instructions are used to take information that goes into the computer (inputs), process that information, and then produce the results that the user wants (outputs).

Think about a calculator app on your phone. You type in numbers (inputs), and the app performs calculations based on its instructions to show you the answer (output). The speed and accuracy of these calculations are part of the 'performance' the user expects.

Software also includes helpful guides, like a user manual, that explain how to use the software and what it can do. This helps users understand and work with the system effectively.

Today's software is comprised of

Imagine building with LEGOs. Software is like that, but instead of plastic bricks, it's made of a few key things working together.

First, there are the actual instructions that tell the computer what to do. Think of these like the recipe a chef follows. These are called **instructions** (or **computer programs**), and when the computer follows them, it performs specific tasks and works the way we expect.

Second, software needs ways to handle information. These are called **data structures**. Imagine a filing cabinet or a well-organized toolbox – data structures are ways to store and manage information so that the programs can use it effectively.

Third, software comes with **documents**. These are like instruction manuals or guides that explain how the software works and how to use it. Some features in the software might be optional, and the documents will explain those too, like choosing extra toppings for a pizza.

Software also needs to know what kind of computer system it will run on. This is called the **platform**. It's like knowing whether your recipe is for a gas stove or an electric stove – you need to know the specifics of the cooking equipment. So, software is often described by what it can do and what kind of platform it needs to run.

Let's also touch on a few other related ideas that are part of the software development journey:

- **Source Code:** This is the original code written by programmers in a language like Python or Java. It's like the rough draft of a book before it's published.
- **Executable:** This is the version of the code that a computer can directly run, without needing the original source code. Think of it as the finished, published book that anyone can read.
- **Design Documents:** Before building software, people create blueprints. These documents outline how the software will be structured, like input and output designs, or diagrams showing how different parts connect (like **UML**, which stands for Unified Modeling Language, a way to visualize software designs).
- **Operations:** These are the expected functions or tasks that the software should perform. It's like listing all the dishes a restaurant should be able to cook.

- **System Manuals:** These are comprehensive guides for users and technical staff, explaining how the software works in detail.
- **Installation:** This is the process of putting the software onto a user's computer so they can use it. It's like setting up a new appliance in your home.
- **Implementation:** This is when the organization starts using the new software in their daily work. They need to make sure it works smoothly with their existing systems, like making sure a new coffee machine doesn't disrupt the workflow in a cafe.

The Term Engineering

You know from our previous chat that software is like a set of instructions that takes some input and gives you a desired output, along with helpful notes. Now, let's look at the word 'engineering' itself.

Think about building a bridge or designing a new car. That's engineering! It's the job or activity of planning how things like roads, bridges, or machines will be built or created, using knowledge to develop them.

When engineers work on something, they have specific goals in mind – like making sure a bridge can hold a lot of weight. But they also have limitations or 'constraints' that affect their work, like how much money they have for the project or the types of materials they can use.

Engineers manage everything by carefully measuring things, taking action, and following a process. They figure out what's needed and gather the necessary resources to meet those goals and create a product or service. This is what engineering is all about.

Interestingly, the word 'engineering' can also mean to make, achieve, or get something through 'contrivance' or 'guile'.

- **Contrivance** means something that's invented or designed for a specific job, like a special machine. It's about clever invention.
- **Guile**, in this context, isn't about being deceitful in a bad way, but more about finding clever, even 'cunning' solutions to problems, especially those that aren't straightforward mechanical issues.

So, engineering is more than just a subject you study; it's an active way of approaching problems. It's a verb – an action word – that describes how engineers get things done.

Source Code vs. Executable in Visual Basic

Let's talk about two different ways we see computer instructions.

Imagine you have a recipe for baking a cake. The recipe itself, written down in a cookbook, is like **source code**. It's the human-readable set of instructions that a chef (or a programmer) can understand and follow. It tells you exactly what ingredients to use, what steps to take, and in what order.

Now, imagine you've followed that recipe and baked the cake. The finished cake is like an **executable**. It's the final product that can be directly used or 'run' without needing to see the original recipe. For computers, an executable is a program that's ready to be run by the computer's processor (the part that does the thinking and calculating).

This section uses Visual Basic, a tool for creating software, to show the difference between these two.

Source Code:

- This is the actual text written by a programmer using a programming language (like Visual Basic).
- It's meant for humans to read and edit.
- Think of it as the blueprint or the recipe.

Executable:

- This is a file that the computer can understand and run directly.
- It's created by translating the source code into a language the computer's processor can execute.
- Think of it as the finished building or the baked cake – ready to go!

So, when you're working with software, you might see the original instructions (source code) or the finished, runnable program (executable).

The Term Software Engineering

You know how building a really big and complicated structure, like a skyscraper, needs a special kind of planning and a careful process? That's where engineering comes in. Software engineering is like applying that same thoughtful, structured approach, but instead of buildings, we're building software.

Think about building a bridge. You wouldn't just start hammering nails, right? You'd need to figure out how much weight it needs to hold, what materials to use, and have a detailed plan to make sure it's safe and strong. Software engineering is similar, but for computer programs, especially the big, complex ones.

It's about using what we know from science and applying it to create software that solves real-world problems in a way that's efficient and doesn't cost too much. This means we analyze what people actually need the software to do, then design it, build it, and test it thoroughly.

At its core, software engineering is about using principles that engineers use for physical things, but for creating software. It focuses on finding the best ways and processes to develop software, aiming for high quality and reliability.

Here's a more formal way to look at it: Software engineering is a systematic, disciplined, and measurable way to develop, run, and maintain software. It's essentially applying the principles of engineering to software.

Computer Science VS Software Engineering

You've learned that engineering is about designing and building things in a structured way, and that software engineering applies these principles to creating software. Now, let's look at two related but distinct fields: Computer Science and Software Engineering.

Think of it like this: Imagine you want to build a fantastic new type of bridge.

Computer Science is like studying the fundamental physics of how materials behave under stress, the mathematical principles of load-bearing, and the theoretical limits of how strong a bridge can be. It's about

understanding the 'why' and the 'what's possible' from a deep, often theoretical, viewpoint.

Software Engineering, on the other hand, is like the actual process of designing and building that bridge. It involves planning the project, managing the workers, ensuring the materials meet safety standards, testing the structure, and making sure it gets built on time and within budget. It's about the 'how' of creating a real-world, functional system.

Here's a breakdown:

- **Computer Science:** This field focuses on the theoretical and mathematical aspects of how computers work. It delves into the foundational ideas behind computation.
- It covers topics like **algorithms** (step-by-step procedures for solving problems) and **data structures** (ways to organize and store data efficiently).
- You might choose Computer Science if you enjoy math and logic, or if you're interested in specialized areas like artificial intelligence (making computers think like humans), machine learning (computers learning from data), security, or computer graphics.
- **Software Engineering:** This field is more about the practical, hands-on process of building and maintaining software systems. It's about the entire journey from idea to a finished, working product.
- It includes topics like **project management** (planning and overseeing software development), **software architecture** (the high-level structure of a software system), **risk management** (identifying and dealing with potential problems), and **quality assurance** and **testing** (making sure the software works correctly and reliably).
- You might choose Software Engineering if you prefer a hands-on approach and want to learn the complete life cycle of how software is created and kept running.

Both fields teach you the basics of programming and computer science, so either can lead you to a career as a software developer. The choice depends on whether you're more drawn to the theoretical foundations or the practical building and management of software.

OR

Think of building a really big, complex LEGO castle. You wouldn't just start sticking bricks together randomly, right? You'd want a plan, specific instructions, and maybe even a team of builders working together to make sure it's strong, looks good, and doesn't fall apart.

Software Engineering is a bit like that for building software. It's about using smart methods and scientific knowledge to create software solutions that are:

- **Practical:** They actually work and do what they're supposed to do.
- **Cost-effective:** They're built without wasting too much time or money.
- **Well-designed:** They're planned out carefully.
- **Constructed:** They're built following that plan.
- **Operated:** They can be used easily.
- **Maintained:** They can be fixed or updated later without too much trouble.

Basically, the goal of Software Engineering is to make sure the software we create is:

- **Fault-free:** It doesn't have many errors or bugs.
- **Satisfies user needs:** It does what the people using it want it to do.
- **On time and within budget:** It gets finished when promised and doesn't cost more than expected.

To achieve this, Software Engineering involves a whole set of movements, methods, and techniques that help make the process of creating software more **systematic**. This means it's more organized, planned, and less like throwing things at a wall to see what sticks. It's about having a clear, repeatable process.

The key differences are

You've learned that Computer Science and Software Engineering are related but have different focuses. Let's look at how they differ, especially when it comes to what you study.

Think of it like choosing between studying general cooking and specializing in professional pastry making.

- **Computer Science:** This field is like a general cooking program. It gives you a broad education with lots of choices. You can explore many exciting areas like making computers understand images (computer vision), teaching them to learn (machine learning), keeping information safe (security), or organizing data (database management). You get to pick many of these advanced topics as electives, which are optional courses you can choose to take.
- **Software Engineering:** This field is more like specializing in professional pastry making. It requires you to take more core courses specifically about building software well. This includes learning how to check if software works correctly (software testing), how to plan and structure software (design), and how to figure out exactly what the software needs to do (software requirements specification). These are fundamental skills for building reliable software.

So, Computer Science offers more flexibility in exploring different advanced computing topics, while Software Engineering has a stronger emphasis on the specific skills needed to build software.

Importance of Software

Software is like the invisible engine that powers so much of our modern world. It's not just a tool; it's become a fundamental driving force.

Think of it this way:

- **Business Decision Making:** Imagine a busy store owner needing to know what products are selling best to decide what to order next. Software helps businesses gather and understand all that information quickly, acting like a super-smart assistant for making important choices.
- **Scientific and Engineering Breakthroughs:** Scientists and engineers use software like specialized microscopes or powerful calculators to explore complex problems. It allows them to run simulations, analyze data, and discover new things in fields like medicine, physics, and building bridges.
- **Embedded Everywhere:** Software is hidden inside so many things we use every day. It's the smart part of your car's navigation system, the logic that makes your phone work, the control system in a hospital's medical equipment, and even what makes your TV channels switch. It's in transportation, communication, healthcare, and entertainment.

- **Affecting Our Lives:** Because software is in all these places, it touches almost every part of our lives – how we shop, how we communicate, what we watch, and how we work. Its influence on our society and culture is really significant.

As software becomes more important, the people who build it are always trying to find better, quicker, and cheaper ways to create high-quality programs that do what we need them to do.